

Alumnos: Citlaly Zeferino

Dafne Pineda

Sebastián Erives

Matricula: T07048221

T07077540

T03004352

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de aplicaciones móviles

Practica 10

Introducción

En esta aplicación la desarrollamos pensando en cómo organizar las actividades diarias de forma mas sencilla y rápida, esta permite al usuario llevar en control claro de sus pendientes.

Funciones principales de la aplicación

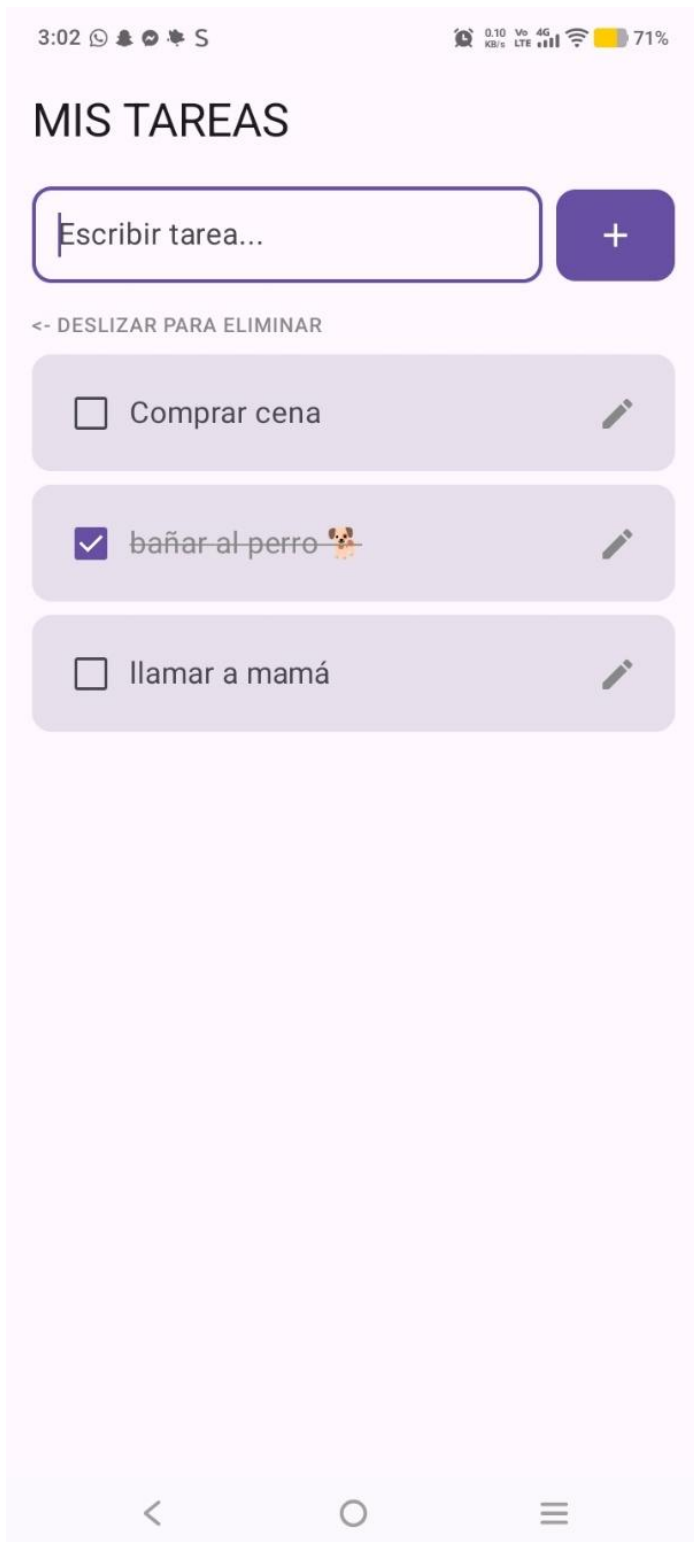
La pantalla principal utiliza variables de estado observable para reaccionar en tiempo real ante cualquier interacción del usuario.

La lista de tareas, la cadena del campo de texto y el puntero de edición están vinculados al ciclo de recomposición de Compose. Al modificar cualquier valor, el motor de Compose repinta únicamente las secciones afectadas de la interfaz sin reiniciar la pantalla.

```
Usage
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun TodoScreen() {
    var tasks by remember {
        mutableStateOf(
            value = listOf(
                Task(id = 1, text = "Bañar al perro", isCompleted = true),
                Task(id = 2, text = "Comprar comestibles para la cena", isCompleted = false),
                Task(id = 3, text = "Llamar a mamá", isCompleted = false)
            )
        )
    }

    var inputText by remember { mutableStateOf( value = "") }
    var editingTaskId by remember { mutableStateOf<Long?>( value = null) }

    val onSaveTask = {
        if (inputText.isNotBlank()) {
            if (editingTaskId != null) {
                tasks = tasks.map { if (it.id == editingTaskId) it.copy(text = inputText) else it }
                editingTaskId = null
            } else {
                tasks = tasks + Task(text = inputText)
            }
            inputText = ""
        }
    }
}
```



Se implementa un comportamiento contextual multimodelo. Si `editingTaskId` es nulo, la lógica ejecuta la inserción de un nuevo elemento dentro del arreglo `tasks`. Si

contiene un id, ejecuta un mapeo para actualizar el registro existente. El ícono del botón cambia dinámicamente

```
OutlinedTextField(  
    value = inputText,  
    onValueChange = { inputText = it },  
    placeholder = { Text(if (editingTaskId != null) "Editar tarea..." else "Escribir tarea...") },  
    modifier = Modifier.weight(1f),  
    shape = RoundedCornerShape(size = 12.dp),  
    singleLine = true  
)
```

3:03

0.00 KB/s 4G LTE 71%

MIS TAREAS

llamar a mamá despues de cenar



<- DESLIZAR PARA ELIMINAR

☐ Comprar cena



☒ bañar al perro 🐕



☐ llamar a mamá despues de cenar



cenar

cenaron

cenaremos



1 2 3 4 5 6 7 8 9 0

q w e r t y u i o p

a s d f g h j k l ñ

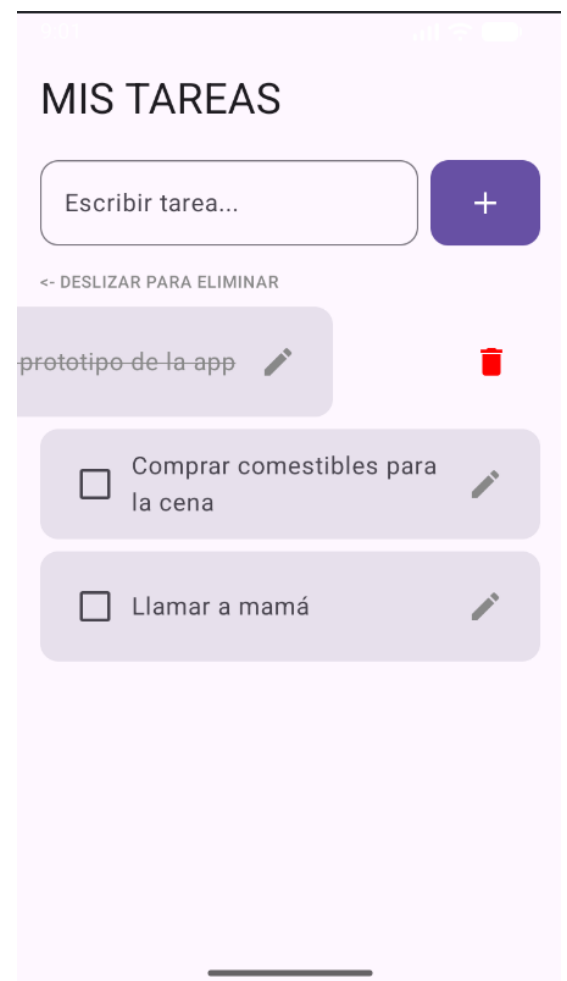
⬆ z x c v b n m ⬇

?123 , 😊 . ✓



Se restringe la dirección del gesto únicamente de derecha a izquierda. La función `animateColorAsState` calcula dinámicamente la transición de color de la capa inferior, cambiando suavemente a rojo a medida que el usuario arrastra la tarjeta. Al completar el recorrido, se ejecuta un filtro sobre el arreglo original para remover el elemento.

```
val dismissState = rememberSwipeToDismissBoxState(  
    confirmValueChange = { dismissValue ->  
        if (dismissValue == SwipeToDismissBoxValue.EndToStart) {  
            tasks = tasks.filter { it.id != task.id }  
            true  
        } else {  
            false  
        }  
    }  
)
```



Conclusión

Esta práctica nos permitió entender mejor cómo trabajar con Jetpack Compose y el manejo de estados en Android. Al vincular directamente las variables observables con la recomposición, comprobamos lo práctico que resulta que la pantalla solo actualice los elementos que realmente cambian, lo que evita recargar vistas completas y hace que la lógica de alternar entre crear o editar una tarea sea bastante directa y ordenada.

Por otro lado, implementar la eliminación mediante deslizamiento y animar el cambio de color con `animateColorAsState` mejoró mucho la respuesta visual de la app al interactuar con las tarjetas. En general, el ejercicio nos ayudó a comprobar cómo las herramientas actuales de Compose facilitan escribir menos código repetitivo y lograr una aplicación funcional, rápida y cómoda de usar.